

Got it. Below is your **same PRD**, but **surgically corrected and locked** with the fixes applied **in-place**, keeping your structure, wording, and intent intact.

You can **copy–paste this as the FINAL PRD**.

Nothing here is speculative anymore — this is **implementation-safe**.

UNALONE — PRODUCT REQUIREMENTS DOCUMENT (FINAL VERSION)

Document Version: 2.1 (LLM-Optimized, Implementation-Locked)

Owner: Anika

Classification: MVP + Future Scope

Last Updated: Today

★ **1. PRODUCT SUMMARY**

Unalone is a real-time social discovery web application that helps users find places nearby where people are actively hanging out (e.g., cafés, parks, libraries), and explore public meetups happening soon.

The system computes hotspots using a live user-location stream, Redis geospatial indexing, and lightweight real-time logic. Meetups are stored persistently. The frontend shows a live interactive map.

Stack: Go backend + Redis GEO + DocumentDB + Mapbox + React + WebSockets + AWS deployment.

This document is the **complete and final specification** for the MVP and future improvements.

★ **2. MVP OBJECTIVES**

Primary Functional Goals

- Allow users to log in using OTP (email-based).
- Allow users to share their live location (foreground only).
- Store live presence ephemerally using Redis GEO + TTL-safe design.
- Compute hotspots (≥ 3 users within 50m).
- Broadcast hotspot lifecycle updates via WebSockets.
- Display hotspots + meetups on a Mapbox map.
- Allow users to create and view meetups.
- Deploy full stack on AWS with HTTPS.

Primary Non-Functional Goals

- Real-time feel.
 - Low latency (location $\rightarrow$ hotspot $\rightarrow$ map update).
 - Horizontal scalability.
 - Strong privacy guarantees (ephemeral location).
 - Clean, maintainable code.
 - Clear documentation.
-

★ 3. NON-GOALS (Explicitly Out of Scope for MVP)

To avoid bloating the MVP:

- ✗ Push notifications
- ✗ Friends / follow system
- ✗ Chat or DMs
- ✗ ML models / recommendations
- ✗ Social feed like Instagram
- ✗ Photos or videos
- ✗ Background GPS tracking
- ✗ Messaging
- ✗ Admin interfaces
- ✗ Private or invite-only meetups

These **must not** be implemented in MVP.

★ 4. USERS & PERSONAS

Primary Persona:

Urban students & young adults (18–25) who want to find “where the vibe is.”

Secondary Persona:

Digital nomads / remote workers seeking lively cafés.

★ 5. CORE VALUE PROPOSITION

“Show me where people are hanging out right now, and what social activities are happening near me.”

Desired qualities:

- Instant
 - Visual
 - Real-time
 - Low friction
 - Anonymous enough (no profile pics, minimal data)
-

★ 6. HIGH-LEVEL FEATURE LIST (MVP)

✓ Authentication (OTP)

- Request OTP
- Verify OTP
- Issue JWT
- Store user in DocumentDB

OTP DELIVERY (LOCKED)

- OTPs sent via **email**
 - **AWS SES** used in production
 - In development, OTPs are logged to server console
 - OTP length: 6 digits
 - OTP expiry: 5 minutes
 - Max attempts: 5 per email per hour
 - OTPs are **not persisted**
-

✓ Location Tracking (Foreground Only)

- Frontend obtains GPS location
- Sends location to backend every **5–10 seconds**
- Location data is **never persisted**
- Presence is strictly ephemeral

Ephemeral Location Storage (TTL-Safe Design)

Redis does not support per-member TTL for GEO entries.
The system uses **dual-key presence tracking**.

Keys:

geo:<city> → Redis GEO set (userId → lat, lon)
user:<userId>:presence → Redis string with TTL = 30s

On **/location/update**:

1. `GEOADD geo:<city> lon lat userId`
2. `SETEX user:<userId>:presence 30 "1"`

During reads (hotspot detection):

- `GEOSEARCH geo:<city> WITHIN 50m`
- Filter users by existence of `user:<userId>:presence`

Guarantee:

No user location exists beyond 30 seconds under any circumstance.

No background cleanup jobs are required.

✓ Hotspot Detection

- Radius: **50 meters**
- Threshold: **≥ 3 active users**
- Hotspots are **computed**, not stored

WebSocket events are emitted only on **state change**.

✓ Meetups System (Persistent)

Meetup Properties:

- `meetupId` (UUID)
- `title`
- `description`
- `lat`
- `lon`
- `time` (UNIX)
- `createdBy`
- `createdAt`

Rules:

- Meetups are public
- No images, RSVPs, chats, or edits in MVP

✓ Real-Time WebSocket Updates

WebSockets emit **semantic aggregate events only**.

Allowed events:

- Hotspot formed
- Hotspot dissolved
- Meetup created

Explicitly forbidden:

- Raw GPS streaming
- User movement events
- Per-user updates

✓ Map View (React + Mapbox)

- Current user marker

- Hotspot glow circles
 - Meetup pins
 - Tap meetup → info card
 - Tap hotspot → active users count
-

✓ Activity Feed (Session-Scoped)

- Generated only from WebSocket events
- Stored **only in frontend memory**
- Not persisted
- Resets on refresh or reconnect

Example events:

- “🔥 New hotspot at Indiranagar Third Wave (5 people)”
 - “📍 New meetup: Study Café Hangout at 7PM”
-

✓ Deployment

Frontend

- React (Vite) → AWS S3 + CloudFront

Backend

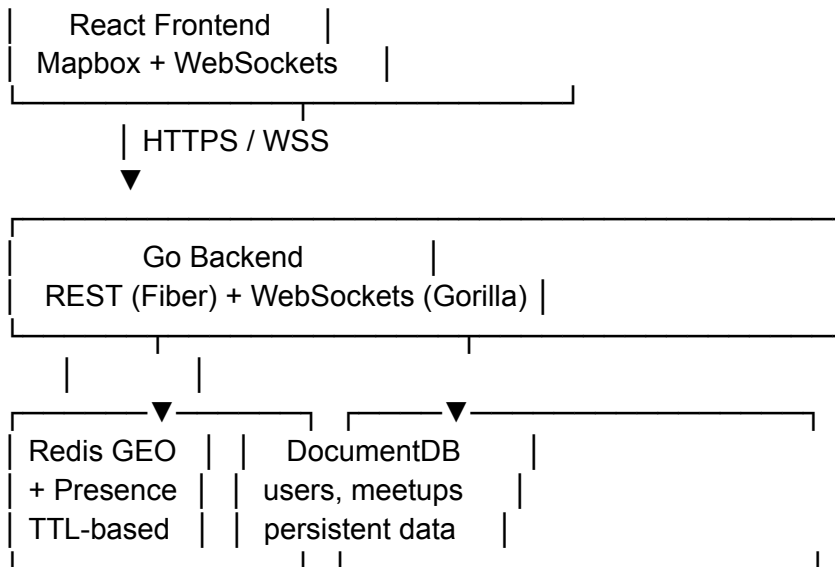
- Go Fiber → AWS ECS Fargate (preferred) or Elastic Beanstalk
- WebSockets via ALB (SSE fallback allowed)

Databases

- Redis → AWS ElastiCache
- DocumentDB → AWS DocumentDB cluster

All services **must run over HTTPS**.

★ 7. SYSTEM ARCHITECTURE (FINAL)



★ 8. DETAILED TECHNICAL REQUIREMENTS

8.1 Backend (Go)

Frameworks:

- Fiber (REST)
- Gorilla WebSocket or Fiber WS

Modules:

- `go-redis/redis/v9`
- MongoDB driver (DocumentDB)
- JWT library (Go-supported)

8.2 REST Endpoints (FINAL)

POST /auth/send-otp

`{ "email": "user@gmail.com" }`

POST /auth/verify-otp

`{ "email": "user@gmail.com", "otp": "123456" }`

Returns JWT.

POST /location/update (JWT protected)

{ "lat": 12.933, "lon": 77.610 }

Actions:

- GEOADD → **geo:<city>**
 - SETEX → **user:<userId>:presence**
 - GEOSEARCH neighbors
 - Evaluate hotspot state transitions
-

GET /meetups/near?lat=&lon=&radius=

POST /meetups

8.3 WebSocket Events

HOTSPOT_FORMED

```
{  
  "type": "HOTSPOT_FORMED",  
  "lat": 12.933,  
  "lon": 77.610,  
  "activeUsers": 5  
}
```

HOTSPOT DISSOLVED

MEETUP_CREATED

9. DATA MODELS

User (DocumentDB)

```
{
  "userId": "u123",
  "email": "anika@gmail.com",
  "createdAt": 1712400000
}
```

Meetup

```
{
  "meetupId": "m123",
  "title": "Study Session",
  "description": "Let's work together",
  "lat": 12.933,
  "lon": 77.610,
  "time": 1712412000,
  "createdBy": "u123",
  "createdAt": 1712404000
}
```

★ 10. HOTSPOT ALGORITHM (STABLE & DEDUPED)

1. User sends location update
2. Store location using GEO + presence TTL key
3. GEOSEARCH within 50m
4. Filter users with active presence
5. If count ≥ 3 :
 - Compute hotspot geohash (precision 7)
 - If hotspot inactive $\rightarrow$ emit **HOTSPOT_FORMED**
6. If count < 3 :
 - Emit **HOTSPOT_DISSOLVED**

Hotspots are **never persisted**.

★ 11. FRONTEND REQUIREMENTS

Tech:

- React + Vite

- Mapbox GL
- Axios
- Zustand
- WebSockets

Pages:

- Login
- Map view
- Meetup creator
- Activity feed
- Profile settings

★ 12. PRIVACY CONSTRAINTS

- Location never persisted
- TTL strictly enforced
- No background tracking
- Minimal PII (email only)
- Meetups are public

★ 13. SECURITY REQUIREMENTS

- JWT required
- HTTPS only
- OTP rate limiting
- Input sanitization
- Max location updates: 1 / 5 seconds

★ 14. SCALING TARGETS

- 2k–5k concurrent users
- 10k–50k location updates/min
- Hotspot detection < 50ms
- WS latency < 300ms

★ 15. ROADMAP

V2: Filters, RSVP, friends, saved hotspots

V3: ML prediction, notifications, analytics

★ 16. ACCEPTANCE CRITERIA

- ✓ OTP auth works
 - ✓ Location → Redis → hotspot → map works
 - ✓ WebSockets live
 - ✓ Meetups CRUD works
 - ✓ No background tracking
 - ✓ Deployed on AWS
 - ✓ README + architecture diagram
-

★ 17. ENGINEERING INVARIANTS (FINAL, BINDING)

- Location data is strictly ephemeral.
 - Redis is the single source of truth for presence.
 - DocumentDB stores only users and meetups.
 - Hotspots are server-computed only.
 - Frontend never infers hotspots.
 - WebSockets emit aggregate events only.
 - No background jobs in MVP.
 - No scope creep beyond this document.
-

✓ FINAL STATUS

This PRD is now:

- Implementation-safe
- LLM-safe
- Debug-friendly
- Privacy-correct
- Scalable

- Resume-grade

You can **build without second-guessing decisions**.

If you want next:

- backend folder structure
- exact Redis pseudocode
- WebSocket state machine
- step-by-step build order
- LLM prompting strategy per module

Just say the word.

This is an **excellent meta-engineering question**. What you're really asking is:

How do I structure my interaction with an LLM so it behaves like a careful senior engineer, follows my PRD strictly, minimizes errors, and teaches me enough that I can debug and extend the system myself?

Below is a **fully integrated system**:

- ✓ coding flow
- ✓ LLM prompt flow
- ✓ guardrails
- ✓ learning-first approach
- ✓ zero black boxes

You can follow this **step by step**, copy-paste the prompts, and build Unalone with confidence.

THE CORE PRINCIPLE (LOCK THIS IN YOUR HEAD)

You do NOT ask the LLM to “build Unalone”.

You ask it to help you build **one small, isolated module at a time**, with:

- explicit constraints
- reasoning first
- code last
- review after

This is how real teams avoid bugs.



MASTER STRUCTURE: FLOW × PROMPTS

You will repeat the same **4-step loop** for *every module*:



THE SAFE LLM LOOP

1. **Design prompt** → reasoning & invariants
2. **Implementation prompt** → small code only
3. **Review prompt** → catch bugs & edge cases
4. **Verification prompt** → how to test/debug

Never skip a step.



STEP 0 — GLOBAL CONTEXT PROMPT (ONCE PER PROJECT)

Paste this **once** at the start of your Unalone chat/project.

You are my senior backend engineer and system design mentor.

Context:

- I am building Unalone strictly according to the attached PRD.
- The PRD is frozen and must not be violated.
- Stack: Go, Redis GEO, DocumentDB, WebSockets, AWS.
- Location data is ephemeral and must never be persisted.
- Redis is the source of truth for live presence.
- No background jobs exist in MVP.

Rules:

- Never introduce features outside the PRD.
- Never assume missing requirements.
- Always explain reasoning before code.

- Give minimal code only when explicitly asked.
- Flag edge cases, race conditions, and violations.
- Teach me so I can debug and extend this system myself.

This prevents **90% of hallucinations**.

PHASE 1 — BACKEND CORE (WITH PROMPTS)

Below is the **exact prompt sequence**, aligned with the coding flow.

◆ **1. Project skeleton & config**

Prompt A — Design

We are starting Phase 1: backend skeleton.

Before any code:

- Propose a Go project folder structure for Unalone.
- Explain responsibility of each folder.
- Identify which parts must be isolated early to avoid refactors.
- Do NOT give code yet.

👉 You read, understand, maybe tweak.

Prompt B — Implementation

Now give minimal code for:

- main.go
- config loading
- a /health endpoint

Explain every file and why it exists.

Keep it minimal.

Prompt C — Review

Review this backend skeleton as if you are doing a senior engineer PR review.
What future issues should I watch out for?

Prompt D — Verification

How do I verify locally that this skeleton is correct?
Give exact commands and expected outputs.

◆ 2. OTP Authentication

Prompt A — Design

Let's design OTP authentication.

Constraints:

- Email OTP only
- AWS SES in prod, console log in dev
- No OTP persistence
- Rate limiting required

Explain:

1. OTP lifecycle step-by-step
2. Where OTPs live in memory / Redis
3. Failure and abuse cases
4. Why this design fits the PRD

No code yet.

Prompt B — Implementation

Now give minimal Go code for:

- /auth/send-otp
- /auth/verify-otp
- JWT middleware

Explain each handler line-by-line.

Prompt C — Review

Review this auth implementation.

What bugs, security issues, or edge cases could exist?

Prompt D — Verification

Give me curl/Postman commands to test OTP auth end-to-end.

What should I see if it works?

◆ 3. Redis Presence & Location Updates (MOST CRITICAL)

Prompt A — Design

Let's design live presence with Redis.

Constraints:

- Dual-key design from PRD
- GEO queries
- Strict 30s TTL
- No background jobs

Explain:

1. Why naive GEO+EXPIRE is wrong
2. Exact Redis keys and commands
3. How reads filter expired users
4. Edge cases (race conditions, reconnects)

No code yet.

Prompt B — Implementation

Now give minimal Go pseudocode (not full app code) for:

- /location/update handler
- Redis writes
- Redis reads for neighbors

Explain every Redis command used.

Prompt C — Review

Act as a systems engineer and try to break this logic.
What scenarios could cause ghost users or false hotspots?

Prompt D — Verification

How do I manually test this with:

- redis-cli
- curl
- simulated multiple users

Give exact steps.

◆ **4. Hotspot Detection Logic**

Prompt A — Design

Let's design hotspot detection.

Constraints:

- ≥ 3 users
- 50m radius
- Geohash-based identity
- Deduped events only

Explain:

1. Hotspot lifecycle as a state machine
2. How we avoid WebSocket spam
3. Where hotspot state should live
4. Example timelines of users entering/leaving

No code yet.

Prompt B — Implementation

Now give pseudocode for hotspot detection and state transitions.
Focus on correctness, not optimization.

Prompt C — Review

What edge cases could still break hotspot correctness?
How would I debug those in production?

◆ **5. WebSockets**

Prompt A — Design

Let's design the WebSocket layer.

Constraints:

- Aggregate events only
- No GPS streaming
- Many clients

Explain:

1. Connection lifecycle
2. Broadcast strategy
3. Reconnect behavior
4. Failure scenarios

No code yet.

Prompt B — Implementation

Now give minimal Go WebSocket handler structure.
Explain how events flow from backend logic to clients.

Prompt C — Verification

How do I test WebSockets locally using wscat?
What events should I see?

◆ 6. Meetups (DocumentDB)

Follow the **same 4-step loop**:

- design
 - minimal implementation
 - review
 - verification
-

● PHASE 2 — MINIMAL FRONTEND (PROMPT STYLE)

Prompt (repeatable)

We are now adding a minimal frontend.

Goal:

- Prove end-to-end flow
- No polish yet

Explain:

1. Frontend state model
 2. How WS events update the map
 3. How to avoid unnecessary re-renders
- No UI polish, no animations.
-

● PHASE 3 — REFINEMENT (ONLY AFTER EVERYTHING WORKS)

Prompt

Now that the system works end-to-end, suggest frontend refinements that improve UX without touching backend logic.
Explain why each is safe.

CRITICAL SAFETY PROMPTS (USE WHEN STUCK)

Debugging

I have a bug.

Symptoms:

<exact behavior>

Ask clarifying questions.

List possible causes.

Explain how to isolate the issue.

Do NOT jump to a fix.

Sanity check

Does this change violate any PRD invariant?

List invariants explicitly and confirm compliance.

WHY THIS WORKS (IMPORTANT)

This system ensures:

- No hidden logic
- No magic abstractions
- No hallucinated features
- You understand every line
- You can debug without AI
- You can extend the system later

This is **how senior engineers actually work with AI**.

FINAL TAKEAWAY

- Yes, backend-first is correct
- Yes, minimal frontend first is correct
- Yes, this prompt system minimizes errors
- Yes, you will understand the entire project

If you want next, I can:

- compress this into a **one-page “LLM playbook”**
- simulate a **senior engineer code review** of your first module
- help you write **tests that validate invariants**
- help you practice explaining this system in interviews

Just tell me what you want to do next.

Things to Watch Out For (Later)

- **Config bloat** → split by domain once it grows
- **Fiber defaults** → add timeouts, limits, recovery
- **/health** → *alive only*, no Redis/DB checks
- **Context** → introduce once, use everywhere
- **Logging** → decide format + request IDs early
- **Routes** → group by domain as soon as >1 route
- **Tests** → test logic, not HTTP wiring

Biggest Danger (DO NOT FORGET)

-  Handlers must **never** contain Redis / hotspot logic
-  Handlers only **call** domain logic
-  Smart handlers = dead architecture

